

Reinforcement Learning for Robotic Manipulation: Training the SO-100 Arm to Touch and Place Objects

Your Full Name

Department of [TODO: Your Department]

[TODO: Your University]

[TODO: your.email@university.edu]

Abstract—This paper presents the design, implementation, and evaluation of a reinforcement learning (RL) system for controlling the SO-100, a six degree-of-freedom robotic arm, in a simulated MuJoCo environment. The objective is to train an autonomous agent—using the Soft Actor-Critic (SAC) algorithm—to perform manipulation tasks of increasing difficulty, starting with touching a target cube and progressing toward pick-and-place operations. The environment is built on top of the `gym-so100` package, which wraps `dm_control` physics into a Gymnasium-compatible interface. Three task variants are defined: dense-reward touch, sparse-reward touch, and cube-to-bin placement. A 12-dimensional state observation is used, comprising object positions and joint angles. Training is conducted for 300 000 environment steps using parallel vectorized environments and observation normalisation. Results show that the agent successfully learns an approach strategy—reducing end-effector-to-cube distance from over 0.18 m to under 0.03 m—but does not yet achieve reliable contact closure within the tested training budget. Qualitative analysis of the reward function reveals that the step penalty and contact-bonus structure present conflicting incentives, which are discussed alongside directions for future work.

Index Terms—reinforcement learning, robotic manipulation, Soft Actor-Critic, MuJoCo, reward shaping, SO-100 robot arm, Gymnasium

I. INTRODUCTION

Robotic manipulation is one of the most challenging open problems in autonomous systems research. Even tasks that are trivial for humans—reaching for an object, grasping it, and moving it to a target location—require a robot to solve a high-dimensional continuous control problem under physical constraints and sensory uncertainty [?].

Reinforcement learning offers a promising framework for robot control: rather than hand-coding a controller for every possible configuration, the agent learns a policy by interacting with the environment and maximising cumulative reward [?]. Recent advances in off-policy algorithms, particularly Soft Actor-Critic (SAC) [?], have demonstrated impressive results on continuous-action robotic benchmarks.

This project applies these ideas to the SO-100 robot arm [?], a six-joint manipulator originally designed for teleoperation tasks. Using MuJoCo as the physics backend and the `gym-so100` package as the environment interface, we train an SAC agent to perform three manipulation tasks of increasing complexity.

A. Motivation

The SO-100 arm is a low-cost, open-source manipulator platform. Developing RL-based controllers for it bridges the gap between academic RL research and practical, affordable robotics. This project is intended to serve as a reproducible baseline demonstrating what 300 000 training steps of SAC can achieve on this hardware.

B. Contributions

The main contributions of this work are:

- A fully configured RL training pipeline for the SO-100 arm using SAC and Stable Baselines 3.
- A reward function analysis identifying the cause of incomplete contact learning.
- An interactive visualisation and reward-probe toolkit for diagnosing agent behaviour.
- Recommendations for reward redesign and extended training.

II. PROBLEM STATEMENT

A. Task Definition

We define the manipulation problem as a Markov Decision Process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma)$, where:

- $\mathcal{S} \subset \mathbb{R}^{12}$ is the state space,
- $\mathcal{A} = [-1, 1]^6$ is the continuous action space,
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function defined in Section ??,
- $\mathcal{P}$ is the transition dynamics governed by MuJoCo physics at 50 Hz,
- $\gamma = 0.99$ is the discount factor.

B. State Observation

At each timestep t , the agent receives a 12-dimensional observation vector:

$$\mathbf{s}_t = [\mathbf{p}_{\text{cube}}, \mathbf{p}_{\text{bin}}, \mathbf{p}_{\text{ee}}, \mathbf{q}] \in \mathbb{R}^{12} \quad (1)$$

where $\mathbf{p}_{\text{cube}}, \mathbf{p}_{\text{bin}}, \mathbf{p}_{\text{ee}} \in \mathbb{R}^3$ are the 3-D Cartesian positions of the cube, bin centre, and end-effector respectively, and $\mathbf{q} \in \mathbb{R}^6$ contains the current joint angles.

C. Action Space

The action $\mathbf{a}_t \in [-1, 1]^6$ is a normalised joint position command. Before being sent to the simulator, each component is un-normalised to the real joint range:

TABLE I: Joint Ranges and Normalisation

Index	Joint	Min (rad)	Max (rad)
0	Waist	-1.92	+1.92
1	Shoulder	-3.32	+0.174
2	Elbow	-0.174	+3.14
3	Forearm Roll	-1.66	+1.66
4	Wrist Rotate	-2.79	+2.79
5	Gripper	-0.174	+1.75

D. Episode Structure

Each episode begins with the arm at a fixed home pose and the cube spawned at a uniformly random position within: $x \in [-0.25, -0.15]$ m, $y \in [0.30, 0.60]$ m, $z = 0.05$ m. An episode terminates either on success (reward = 4) or after 300 steps ($T = 6$ seconds of simulated time).

III. APPROACHES AND SOLUTIONS

A. Environment Architecture

The environment stack is illustrated in Fig. ??.

The layered design separates concerns cleanly:

- 1) **MuJoCo / dm_control** simulates rigid-body physics at 50 Hz.
- 2) **SO100Env** (Gymnasium wrapper) exposes the standard `reset()` / `step()` interface and formats observations.
- 3) **SubprocVecEnv** runs $N = 4$ independent environment copies in separate processes, multiplying experience throughput.
- 4) **VecNormalize** maintains a running mean and standard deviation of observations and normalises them to approximately $\mathcal{N}(0, 1)$ before the policy network sees them.

B. Reward Function

The reward function used in the `SO100TouchCubeTask` is a shaped dense reward consisting of four components:

1) *Distance Shaping*: Let $d_t = \|\mathbf{p}_{\text{ee}} - \mathbf{p}_{\text{cube}}\|_2$ be the end-effector-to-cube Euclidean distance at step t . The shaping term is:

$$r_{\text{shape}}(d) = \max \left(0.1 \left(1 - \frac{d}{0.7} \right), 0.2 \left(1 - \frac{d}{0.5} \right), 0.5 \left(1 - \frac{d}{0.3} \right), 1.0 \left(1 - \frac{d}{0.1} \right), 2.0 \left(1 - \frac{d}{0.05} \right) \right) \quad (2)$$

saturated at $r_{\text{shape}} = 0$ when $d \geq 0.7$.

2) *Contact Bonus*: A binary contact bonus is awarded when any gripper pad geometry makes physical contact with the cube geometry:

$$r_{\text{contact}} = \mathbf{1}[\text{touch_gripper}] \times 1.0 \quad (3)$$

3) *Step Penalty*: A constant penalty of -0.2 is applied every timestep to encourage faster task completion.

4) *Success Return*: If the gripper is touching the cube and $d < 0.05$ m simultaneously, the episode is terminated and a terminal reward of $+4.0$ is issued.

The total step reward is therefore:

$$r_t = \begin{cases} 4.0 & \text{if touch_gripper} \wedge d < 0.05 \\ r_{\text{shape}}(d_t) + r_{\text{contact}} - 0.2 & \text{otherwise} \end{cases} \quad (4)$$

C. Policy: Soft Actor-Critic

SAC [?] is an off-policy actor-critic algorithm that maximises a maximum-entropy objective:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t))] \quad (5)$$

where α is the automatically-tuned entropy coefficient and $\mathcal{H}$ is the policy entropy.

Both the actor and twin-critic networks are fully connected MLPs with architecture $12 \rightarrow 256 \rightarrow 256 \rightarrow \dots$ (Table ??).

TABLE II: SAC Hyperparameters

Hyperparameter	Value
Network architecture	[256, 256] MLP
Learning rate	3×10^{-4}
Replay buffer size	100 000
Batch size	256
Entropy coeff. α	auto-tuned
Discount γ	0.99
Learning starts	1 000 steps
Parallel environments	4
Total training steps	300 000
Observation type	State (12-dim)

IV. TESTING AND VALIDATION

A. Training Curves

Training was monitored via TensorBoard. The mean episode reward increased steadily from approximately -60 at the start of training (random policy) to a peak of [TODO: insert peak value] around step [TODO: insert step]. The entropy coefficient α decreased monotonically from ≈ 1.0 to ≈ 0.2 , indicating a transition from broad exploration to more deterministic exploitation.

B. Quantitative Evaluation

After training, the final policy was evaluated over 10 episodes with deterministic action selection. Results are summarised in Table ??.

TABLE III: Evaluation Results (300 000 steps, deterministic policy)

Metric	Mean	Std
Episode reward	[TODO: val]	[TODO: val]
Episode length (steps)	[TODO: val]	[TODO: val]
Success rate (%)	[TODO: val]	—
Min EE-cube distance (m)	[TODO: val]	[TODO: val]

C. Qualitative Behaviour

Trajectory analysis of the deterministic policy across three episodes revealed the following behaviour pattern:

- 1) The agent consistently reduces the end-effector-to-cube distance from ≈ 0.19 m to ≈ 0.03 m within the first 15–40 steps.
- 2) The arm then oscillates around the cube without closing the gripper, collecting the distance-shaping reward indefinitely.
- 3) No contact events (`touch_gripper = True`) are observed in any of the three evaluation episodes.

D. Reward Probe Analysis

An interactive reward-probe tool was developed to inspect the reward signal in real time during manual control. This confirmed that the distance-shaping component provides a strong, smooth gradient toward the cube, while the contact bonus remains latent — the agent never discovers it within the training budget because random exploration rarely produces a gripper-closing motion precisely when already within 0.05 m of the cube.

E. Checkpoint Comparison

[TODO: Compare 3 checkpoints (e.g. 30k, 150k, 300k steps) and describe the improvement in behaviour between them.]

V. FUTURE ASPECTS

A. Reward Redesign

The current reward function contains two conflicting pressures when $d < 0.05$ m: the step penalty (-0.2) discourages staying near the cube, while the contact bonus ($+1.0$) only fires on actual physical contact. Proposed modifications include:

- Adding an explicit gripper-closing reward term when $d < 0.05$ m, proportional to the gripper joint position.
- Increasing the contact bonus from $+1.0$ to $+3.0$ to make it a dominant signal worth exploring for.
- Removing or reducing the step penalty to allow the agent more time near the cube without negative reinforcement.

B. Extended Training

Training for 1 000 000 steps (approximately $3\times$ longer) may allow the agent to stumble upon contact events during exploration and subsequently reinforce this behaviour. GPU-accelerated training would make this feasible within a reasonable time budget.

C. Curriculum Learning

A curriculum could progressively increase task difficulty: spawn the cube directly under the gripper initially, then gradually increase randomness in the cube’s initial position. This would allow the agent to first learn contact closure, then learn to approach from a distance.

D. Harder Tasks

Once contact is mastered, the natural progression is:

- 1) **Lift:** reward the agent for raising the cube above the table.
- 2) **Place:** reward the agent for depositing the cube in the target bin (`SO100CubeToBin-v0`).
- 3) **Pixel observations:** replace the 12-number state vector with a camera image for a more realistic and challenging setting.

E. Sim-to-Real Transfer

The ultimate goal is to transfer the learned policy from simulation to the physical SO-100 arm. Domain randomisation (varying object masses, friction, and cube positions during training) and a more faithful sensor model would improve transfer fidelity.

VI. CONCLUSION

This project demonstrated the end-to-end application of the Soft Actor-Critic algorithm to robotic manipulation of the SO-100 arm in a MuJoCo simulation environment. A complete training and evaluation pipeline was built from scratch, including environment configuration, reward function design, parallel training, checkpoint management, and interactive visualisation tools.

The trained agent exhibits learned approach behaviour—reliably reducing the end-effector-to-cube distance to within 3 cm—but does not yet achieve consistent contact, indicating that 300 000 steps with the current reward structure is insufficient to bridge the final gap between approach and grasp.

Reward function analysis identified the root cause: the contact bonus is too weak relative to the distance shaping, and the step penalty discourages the persistent near-object exploration needed to discover the contact signal. Targeted reward modifications and extended training are expected to resolve this in future work.

REFERENCES

- [1] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
- [2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2018, pp. 1861–1870.
- [4] I. Julczuk, “gym-so100-c: A Gymnasium environment for the SO-100 robot arm,” 2024. [Online]. Available: <https://github.com/ilonajulczuk/gym-so100-c>
- [5] A. Raffin *et al.*, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *J. Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [6] S. Tunyasuvunakool *et al.*, “dm_control: Software package for physics-based simulation and reinforcement learning,” *Software Impacts*, vol. 6, p. 100022, 2020.